PASADHI Fireworks Inference Evaluation Evidence Audit and Full White Paper

PASADHI contributors | Full White Paper v1.0 | 28 October 2026

Abstract—This paper audits the supplied PASADHI Fireworks test results from 28 October 2026. Fifteen canonical runs contain 120 unique request records: 20 deployment-comparison attempts, 50 live-configured context/load attempts, and 50 simulated requests. The deployment comparisons produced 19 nonempty model responses and one unsupported/empty-response error. In the largest comparison, eight requests per model at shared concurrency two yielded mean latencies of 2,974.07 ms for Muse Glimmer 30B and 765.09 ms for Llama 3.1 8B Instruct; both completed eight of eight requests. All 50 live-configured context/load attempts were rejected by the local 16,000-character input validator before HTTP dispatch, so they do not measure provider context capacity or inference throughput. The simulated records contain deterministic assigned latencies and no model execution. We reconcile duplicate exports, recompute summaries from raw records, examine output excerpts and token accounting, and identify instrumentation limits. The evidence supports a small live integration comparison and a concrete harness-validation finding, not a production benchmark, a long-context capability result, or evidence of model training.

Keywords—PASADHI; Fireworks; inference evaluation; long-context validation; reproducibility; simulation

I. Introduction

PASADHI investigates how office compute and hosted model endpoints can support practical AI workflows. A useful evaluation must establish which backend actually executed, what work was performed, and whether the reported latency or throughput corresponds to useful output. This paper focuses on the result folder supplied by the project owner, rather than extending earlier Docker GEMM experiments or initiating new provider calls.

The immediate application is text cleanup: correcting noisy English and Thai office notes while preserving names, deadlines, order identifiers, quantities and amounts. A short exact-response probe accompanies those examples. This is a plausible integration workload, but the retained records are small and do not include an independently labelled quality benchmark. A nonempty response is therefore treated as transport/application success, not proof that the content is correct.

The central finding is a separation of evidence. Some requests reached deployed models and returned text. A second family was configured as live but stopped at a local input check. A third family generated synthetic records. Combining those families into one performance score would obscure both the useful integration evidence and the failed long-context measurement design. We retain the failures and simulation results as engineering evidence, with their actual scope stated throughout.

II. Evidence Corpus and Provenance

The primary source is test-result/firework/ [1]. Its _work directory contains 15 run folders, each with config.json, dataset.jsonl, run.log.jsonl and summary.json. Three timestamped JSON exports at the folder root reproduce the deployment-comparison requests. Those exports contribute 20 duplicate request representations; they are not additional experiments. Canonical counting uses the _work datasets and run identifiers.

TABLE 1. EVIDENCE CLASSES AFTER DEDUPLICATION

Class	Runs	Requests	Observed outcome
Deployment comparison	3	20	19 model texts; 1 error
Live-configured context/load	6	50	50 local validation errors
Offline simulation	6	50	50 synthetic records

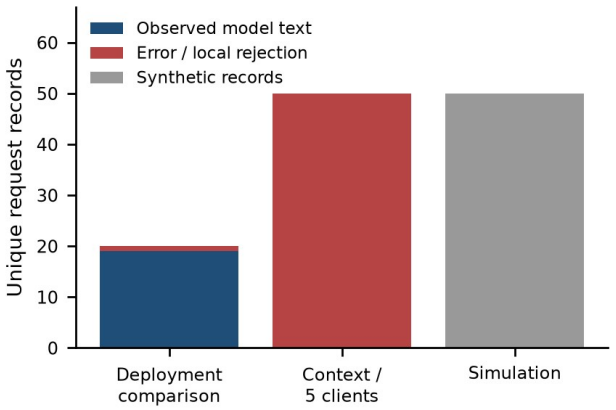


Fig. 1. Evidence categories after removing duplicate exports. Simulated acceptance is distinct from an observed model response; errors include one comparison failure and 50 local rejections.

Every request row was matched to the request events in its run log. Run counts, success/error counts, successful-token totals, per-cell means and nearest-rank percentiles were recomputed and compared with stored summaries. Sequence identifiers were unique within each run. All three root exports matched the corresponding canonical datasets. SHA-256 fingerprints of supplied files are retained in the derived analysis manifest [2].

The current app/hardtest.py and app/fireworks.py implementations were inspected to interpret validation, scheduling, simulation and resource metrics [3], [4]. Their hashes describe the source available during this audit. The input records do not include a run-time Git commit, provider hardware inventory or complete client environment manifest. The code inspection is therefore corroborating

retrospective evidence, not proof of an immutable historical software build.

The comparison exports contain local start times without a timezone suffix. Their epoch timestamps agree with Asia/Bangkok conversion: the principal run starts at 14:45:19 on 28 October 2026, equivalent to 07:45:19 UTC. The paper date and human-readable run timeline use Bangkok time. Source filenames and epoch values are preserved unchanged in the analysis.

III. Experimental Protocol Recovered from the Records

A. Deployment comparison

The comparison targets identify accounts/fireworks/models/muse-glimmer-30b and accounts/fireworks/models/llama-v3p1-8b-instruct, each with a deployment suffix. We refer to them as Glimmer and Llama. The exports call these dedicated deployments, and the identifiers support that routing intent. This audit did not query the provider control plane, so replica counts, GPU type, placement, deployment readiness and isolation cannot be independently established.

TABLE 2. DEPLOYMENT COMPARISON RUNS

Run time	Requests/ model	C	Max output
14:40:47	1	1	128
14:44:54	1	1	1024
14:45:19	8	2	1024

The four-prompt set consists of a short PING instruction, noisy English notes, noisy Thai notes and a mixed-language purchase-order line. The principal run has two observations for each prompt for each model. In the two smoke runs, each model receives only one prompt: Glimmer receives English and Llama receives Thai. Those smoke results are consequently not same-input model comparisons.

The harness constructs jobs by server/model and submits them to one ThreadPoolExecutor. The concurrency value is a shared client-worker count, not an independently enforced per-model load. The job list places the Glimmer group before the Llama group. Prompt assignment follows the request sequence modulo four. In this trace the main run has sequence identifiers 1–8 for Glimmer and 9–16 for Llama, introducing an execution-order confound despite balanced prompt counts.

B. Context and five-client probes

Five nominal context labels are retained: 6K, 16K, 64K, 256K and 600K. Their actual input lengths are counted in characters from config.json, not tokenized with the target model. Ten requests are configured at each of the first four sizes and five at 600K. A further five-client step reuses the smallest long prompt for five requests. All use concurrency five and max_tokens=32; the corresponding simulated runs mirror their inputs and counts.

TABLE 3. NOMINAL CONTEXT VERSUS MEASURED INPUT SIZE

Label	Characters	Live attempts
6K	24064	10
16K	64027	10
64K	256000	10
256K	1024072	10
600K	2400012	5
Five-client step	24064	5

The target label for these live-configured steps is glm-5p3-flash. This is a different target from the Glimmer/Llama comparison. A filename such as fw-serverless or an honesty block marked live indicates configuration intent; it does not by itself establish successful network dispatch. The actual error records and validation path determine the execution classification used here.

IV. Metric Definitions and Audit Rules

Successful live latency is the backend elapsed time around request construction, HTTP execution, payload parsing and text extraction, as recorded in elapsed_ms and propagated to latency_ms. It includes network and provider scheduling effects but does not isolate them. It is not time to first token, decoding-only time, or end-user task completion time. Failed requests are stored with latency_ms=0.0 by the exception handler; those zeros are treated as unavailable measurements rather than instantaneous execution.

Nearest-rank percentiles are recomputed as the sorted value at index ceil(pn), using one-based indexing. For n=8, both p95 and p99 select the largest observed latency. Their equality is a consequence of sample size and estimator, not evidence that the production tail is stable. With n=1, every percentile collapses to the same observation; such rows are smoke checks rather than distribution estimates.

Two throughput definitions are required. Attempt completion rate is all completed records divided by elapsed run time. Successful completion rate is nonempty successful live outputs divided by that time. The stored totals.rps and throughput buckets count all finished records, including errors. Per-cell rps counts accepted outputs but uses the common run duration. Neither measure is a per-deployment saturation throughput estimate in this shared, ordered workload.

For rates derived in this paper, elapsed time is ended_at minus started_at from the raw summary timestamps. Displayed elapsed_s is rounded to two decimal places and is not used to reconstruct sub-second rates. No error-duration distribution is imputed. Tokens are the provider usage fields retained on successful responses, and reasoning tokens are treated as a subset of completion tokens when those fields are present.

V. Live Deployment Results

TABLE 4. PRINCIPAL RUN LATENCY WITH EIGHT SUCCESSES PER MODEL

Metric (ms)	Glimmer	Llama
Mean	2974.07	765.09
p50	2740.29	534.50
p95 / p99	5346.19	1273.98
Observed minimum	1239.49	400.10
Observed maximum	5346.19	1273.98

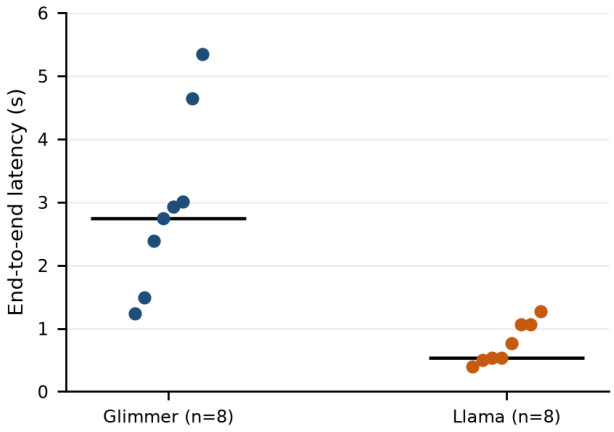


Fig. 2. Individual successful latencies in the principal live comparison. Each dot is one request; the black line is the nearest-rank median. Eight observations per model do not define a reliable production tail.

The principal run completes 16 of 16 requests with a stored elapsed time of 15.11 s. Glimmer has a mean latency 3.89 times Llama's in this trace. This is a descriptive ratio of sample means, not a general speedup guarantee: models differ in size, token accounting, reasoning behavior and output content, while deployment resources and request order are uncontrolled.

Both model cells have a reported rps of 0.53 because each returns eight responses within the same overall run window. That equality does not contradict the latency difference: it is imposed by the balanced request counts and common denominator. Aggregate attempt and successful completion rates are both approximately 1.06 requests/s for this run, with no observed errors.

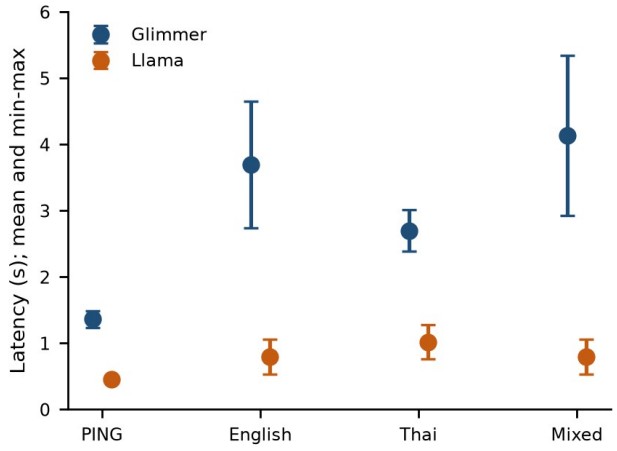


Fig. 3. Latency by prompt category in the principal comparison. Points are means of two requests; bars span the observed minimum and maximum, not SD or confidence intervals.

Prompt-stratified observations show a similar direction across all four categories. Glimmer's mean is 1.365 s for PING, 3.695 s for English, 2.701 s for Thai and 4.136 s for mixed text. Llama's corresponding means are 0.452, 0.794, 1.018 and 0.797 s. With only two examples per category, these values describe the supplied trace and do not support significance tests or a population-level ranking.

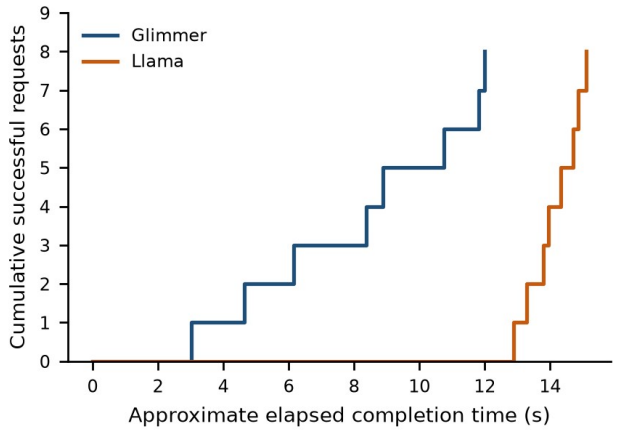


Fig. 4. Approximate cumulative completions from request timestamps plus recorded latency. The sequence illustrates the ordered model groups and shared concurrency; timestamps do not expose an independent provider-side queue.

The completion trace makes the ordered execution visible. The two endpoints are not driven by independent stationary arrival streams, and the later Llama requests encounter a different moment in the run. An external load generator, randomized/interleaved request order, repeated sessions and deployment metadata would be needed before attributing all observed differences to model architecture.

VI. Token Accounting and Output Budget

TABLE 5. PRINCIPAL RUN REPORTED USAGE TOTALS

Usage field	Glimmer	Llama
Prompt tokens	814	648
Completion tokens	3339	233
Reasoning subset	3063	0
Total tokens	4153	881
Other completion	276	233

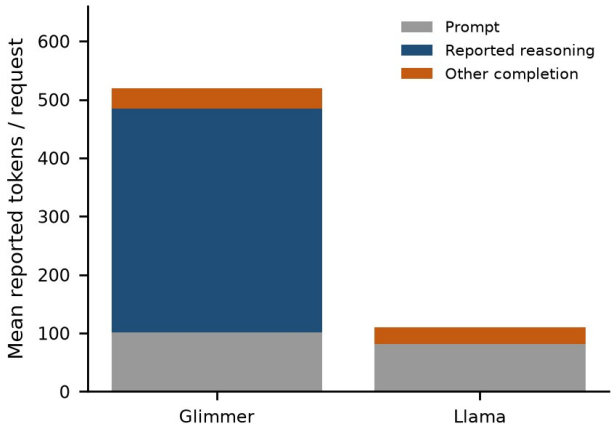


Fig. 5. Mean reported tokens per successful request. Reasoning is shown as a subset of completion tokens, with other completion tokens calculated by subtraction. A missing/zero reasoning field is not proof of an identical internal computation path.

Glimmer reports 3,063 reasoning tokens out of 3,339 completion tokens, or 91.7% of completion usage, in the principal run. Llama records zero reasoning tokens in the corresponding field. The remaining completion counts are 276 and 233 respectively. These are accounting fields, not character counts, visible-word counts or directly comparable computational work across tokenizers.

The first smoke run uses max_tokens=128 and yields an empty-or-unsupported-text error for Glimmer. At max_tokens=1024 in the next smoke run, Glimmer returns nonempty English text in 6,336.41 ms, reporting 745 completion tokens, of which 711 are reasoning tokens. Llama returns Thai text in 704.73 ms. The larger budget therefore accompanies a successful Glimmer response, but the failed response body, usage and finish reason were not retained.

A budget exhausted by reasoning is a plausible hypothesis for the 128-token failure, not a demonstrated cause. Other payload/content-handling issues remain possible. The records provide one failed low-budget observation and no controlled replication. Future instrumentation should retain sanitized finish reasons, provider request identifiers and complete usage for failures without exposing credentials or private text.

No prices, invoices, GPU billing durations or authenticated quota snapshots are included. This paper makes no monetary cost estimate. Dividing total usage tokens by wall time would mix prompt processing, reasoning, generated text and scheduling; it is not presented as model decoding tokens per second.

VII. Qualitative Text Inspection

Output inspection is limited to the retained excerpts, which are capped at 280 characters by the harness. There is no reference-answer file, blinded assessment protocol, language-normalization specification or independent evaluator. The following observations are examples, not an accuracy score or a claim that one model is categorically better at Thai.

The English examples from both models correct visible errors such as “Teh”, “meetng” and “shipmnt”, while retaining Mali, Somchai, Friday and tomorrow. Glimmer joins clauses with a comma; Llama varies punctuation and tense. These differences may be stylistic, and no scoring rubric in the supplied run makes them pass/fail conditions.

For the Thai examples in the principal run, Glimmer’s retained excerpts express the Friday deadline with the standard Friday spelling. Llama’s two excerpts preserve malformed deadline wording and, in one case, a malformed stock spelling. These visible defects matter for the office-note use case, but two observations do not measure a language-wide error rate. A future dataset should label dates and entity preservation separately from orthographic cleanup.

The mixed purchase-order excerpts retain PO#4471, 120 pieces, a shortage of two, the 48,500-baht amount and an undecided shipping date. Glimmer largely preserves the mixed-language form; Llama translates some Thai fragments into English. Whether that translation is desirable requires an explicit policy: a cleanup request alone does not define a target language or forbid translation.

For the PING prompt, Glimmer’s two excerpts are “PING” and Llama’s are “PING.”. The prompt wording includes punctuation around the requested response, and the harness checks nonempty text rather than exact-match compliance. The stored ok flag therefore cannot be reinterpreted as an exact-output or semantic-correctness score.

VIII. Long Context Probes and the Local Validation Boundary

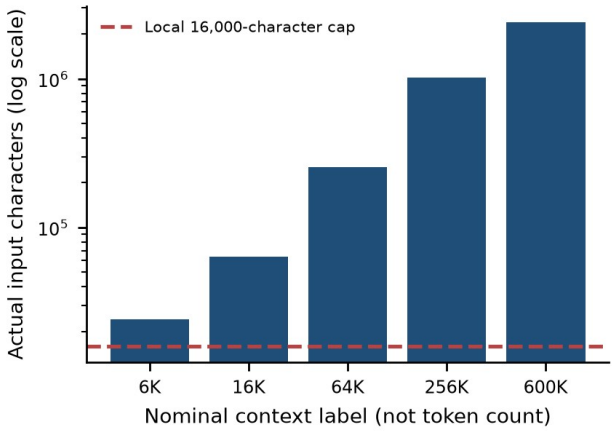


Fig. 6. Actual prompt character lengths versus the local validation cap. The x labels are nominal experiment names; no tokenizer measurement supports interpreting them as verified model token counts.

All five nominal context inputs exceed 16,000 characters. Even the smallest is 24,064 characters, approximately 1.50 times the cap. The largest contains 2,400,012 characters. The FireworksBackend.complete path raises ValueError for inputs above the cap before request construction and urllib.request.urlopen [4]. All 50 live-configured context/load records contain that exact validator message.

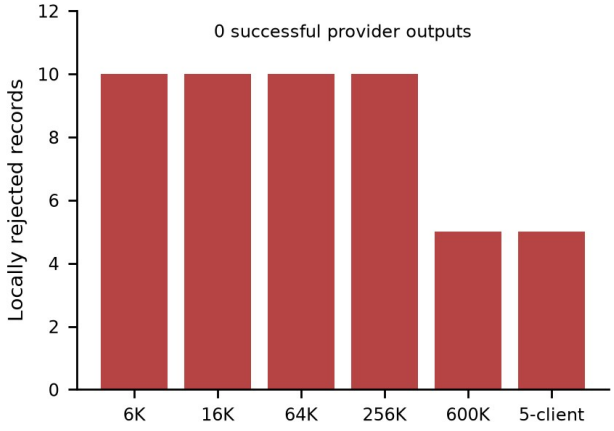


Fig. 7. Rejected records in each live-configured step. All errors share the local prompt-length validation message. These bars are not provider HTTP failures or context-window measurements.

These runs therefore establish a local application boundary. They do not establish that Fireworks accepted a 6K-labelled request, rejected a 600K-token request, completed long-context reasoning, or saturated a model endpoint. No HTTP response code, provider request ID, successful output or positive usage count is retained for these attempts. The generic live honesty string in their summaries overstates execution if read without the request-level errors.

Attempt completion rates in the stored summaries reach 406.17 records/s for one failed step. That rate describes rapid local validation/recording, not inference. The corrected successful live output rate for every such step is zero. Similarly, zero-valued percentile fields with n=0 denote an empty successful sample; they are missing data, not a zero-millisecond latency distribution.

The five-client step uses the same 24,064-character prompt and fails for the same reason. It adds five validation-error observations, not a live concurrency result. A model-specific tokenizer, validated provider limits, configurable application limits and verified network dispatch are prerequisites for a useful future context experiment. Merely increasing a label or deleting a guard would not establish a valid protocol.

IX. Simulation Is a Wiring Test

The six simulated runs produce 50 accepted synthetic records with no provider execution and zero reported tokens. Source inspection shows a deterministic assignment: latency_ms = 40 + ((37 × sequence) mod 120). The simulator returns that number immediately; it does not sleep for the assigned latency. Its numerical output is intentionally a stand-in for a live backend, not a wall-clock model measurement.

TABLE 6. SYNTHETIC LATENCY VALUES DESCRIBE THE GENERATOR

Requests/run	Mean ms	p50 ms	p95 ms
10	99.5	96	151
5	103.0	105	151

The ten-request runs share identical latency summaries across 6K, 16K, 64K and 256K labels because the formula depends on sequence, not input context. The two five-request runs share the second row above. Differences in their wall-clock record rates arise from fast local orchestration and recording, not from model speed or a long-context scaling law.

Assigned simulated latency and wall-clock simulation duration must not be combined as if they measured the same process. For example, several runs finish their full record set in roughly 0.02 s even though each assigned number is tens to hundreds of milliseconds. Reporting a “local model speedup” against real hosted inference would therefore be invalid. Simulation is useful for verifying schema, progress display and failure-free result plumbing only.

X. Resource Sampling and Measurement Coverage

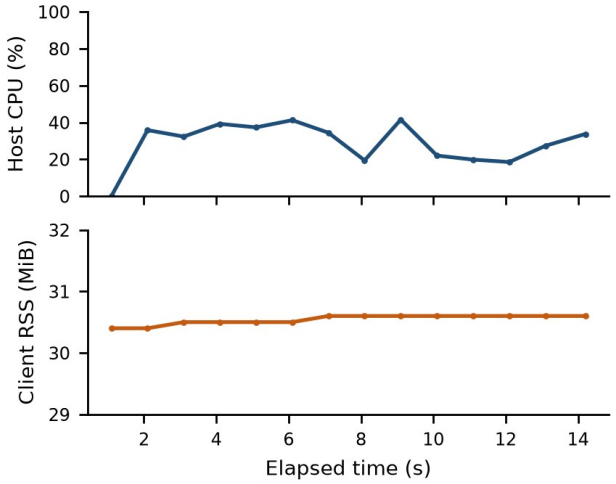


Fig. 8. Resource samples from the main live comparison. CPU is the client host's system-wide psutil.cpu_percent value; RSS is the benchmark process's resident memory divided by 1,048,576. Neither describes Fireworks GPU memory or utilization.

The main live run retains 14 samples, with process RSS between 30.4 and 30.6 MiB and sampled host CPU values between 0.0% and 41.5%. The first CPU value is an initial nonblocking psutil sample and should not be interpreted as an idle-period measurement. The code combines a system-wide CPU reading with a single-process RSS reading; the source resource_note alone does not communicate that distinction.

No resource samples are present in the 12 context/simulation steps. Their note says no samples (psutil not installed), but the current sampler can also return no sample when a very short run ends before its sampling interval. The retained records alone cannot distinguish package absence from timing or another sampling exception. Missing resource data are not converted into zero utilization.

The corpus contains no provider GPU model, VRAM consumption, power measurement, worker/coordinator container inventory, model-loading time or capacity allocation. The earlier Docker fleet benchmark is a different workload and is not merged into these inference results. No distributed multi-node model-execution claim follows from the present files.

XI. Threats to Validity

A. Sample size and ordering

The principal comparison has one session, eight requests per model and two observations per prompt. Smoke tests have one observation per model with different inputs. There is no repeated randomization, steady-state warmup protocol, power analysis or independent deployment replication. Error bars in the prompt plot are observed ranges, and no confidence intervals or significance claims are inferred from them.

B. Response success and quality

The ok flag denotes nonempty extracted content or a synthetic success. It does not require a factual, correctly spelled or instruction-compliant answer. Excerpts may omit trailing content; missing finish reasons prevent robust truncation analysis. Neither a scalar token total nor transport success is a task-quality evaluation.

C. Instrumentation and provenance

Exceptions overwrite latency with zero and discard most failure payload evidence. Summary labels conflate intended live mode with actual execution. Throughput buckets include errors, while per-cell rates use a common elapsed denominator. Exact run-time source revision, client hardware, provider capacity, sampling availability and request warmup state are not captured. These deficiencies bound what a retrospective report can conclude.

D. Scope of the supplied evidence

The files contain inference requests and simulated workload records, not training jobs. There is no SFT/DPO/RFT objective, training loss, evaluation checkpoint, held-out split, model weight update or deployment-change record. Fireworks training skills installed elsewhere in the project do not turn these results into fine-tuning evidence. Likewise, nominal context labels cannot substitute for tokenized inputs and dispatched provider requests.

XII. Engineering Recommendations and Next Experiments

First, separate execution intent from execution phase. Retain fields for configured mode, backend kind, validation outcome, dispatch status, HTTP status, provider request ID and nonempty output. Label a request "provider executed" only when supported by dispatch/response evidence. Start an end-to-end timer before validation and preserve failure duration separately from provider latency.

Second, make attempted-record rate and successful-model-output rate explicit in the schema and dashboards. Use unavailable/null latency statistics when n=0. Keep failed-attempt accounting, accepted responses and simulated records distinct, and ensure dashboards do not plot absent inference latency as zero or simulated assigned latency as observed duration.

Third, redesign context validation around a documented application limit and a model-specific tokenizer. Test boundary-adjacent inputs, confirm a real request was dispatched, collect usage and verify a context-dependent answer. Control request count and spending before increasing input size. Evaluate retrieval or evidence-location tasks rather than relying only on a constant DONE marker.

Fourth, interleave or randomize equal prompt sets across model targets, repeat sessions, capture endpoint resources and report both latency and task quality. Define a Thai cleanup rubric covering spelling, dates, names, numeric fields, missing information and language preservation. Keep a held-out set distinct from demonstration prompts. Record

an explicit output-budget policy for reasoning models and retain sanitized finish reasons.

Finally, treat any fine-tuning proposal as a separate prospective study. Establish a validated baseline and a labelled dataset before selecting a training method. No new inference, deployment, training or billing action was performed to create this white paper; all recommendations here are future work, not completed experiments.

XIII. Reproducibility and Evidence Index

The analysis script reads only the supplied files, deduplicates root exports against canonical datasets, validates log agreement and recomputes summary statistics. The derived outputs include analysis.json, run-summary.csv, canonical-requests.jsonl, eight PNG/SVG plots, this manuscript source, and package/render verification. Inputs are fingerprinted and left unchanged. The report omits credentials and does not read or publish project .env files.

python paper/fireworks-v1.0/analyze.py
python paper/fireworks-v1.0/manuscript.py
[Document-runtime Python] paper/fireworks-v1.0/build_docx.py

The first command re-audits evidence and generates figures, the second regenerates the manuscript structure, and the third clones the retained ResearchPaperTemplate. The final DOCX is rendered to PDF and inspected page by page. Rendering requires the documented Word/LibreOffice workflow and compatible Times fonts. Derived files are a report of existing evidence, not a request to rerun the metered comparison harness.

TABLE 7. RUN INVENTORY

Run identifier suffix	Class	N	OK
compare-144047	Deployment	2	1
compare-144454	Deployment	2	2
compare-144519	Deployment	16	16
6K context	Local error / sim	10 / 10	0 / 10
16K context	Local error / sim	10 / 10	0 / 10
64K context	Local error / sim	10 / 10	0 / 10
256K context	Local error / sim	10 / 10	0 / 10
600K context	Local error / sim	5 / 5	0 / 5
Five-client step	Local error / sim	5 / 5	0 / 5

In the inventory, comparison identifiers abbreviate the date prefix compare-20261028-. Each context row represents two separate runs sharing the context label: a live-configured local-error run and a simulated run. Full step identifiers, character lengths, mode, request counts and unrounded elapsed durations are retained in the machine-readable analysis [2].

XIV. Conclusion

The supplied results establish a working small-scale Fireworks inference path for the deployed Glimmer and Llama targets, including 19 nonempty texts across three comparison runs. Llama has lower observed latency in the balanced principal session, while Glimmer shows different reasoning-token usage and cleaner retained Thai deadline wording in these examples. Neither observation establishes a universal model ranking.

The more decisive methodological finding is that all 50 live-configured long-context/load attempts stopped at the local character validator. They supply no provider context-capacity or long-context performance evidence. The 50 simulated records verify local result plumbing only. A useful next evaluation must instrument execution phase, preserve failed-request timing, tokenize real inputs, randomize matched workloads and score task quality independently of transport success.

References

[1] PASADHI project, supplied result corpus, test-result/firework/, 28 October 2026. Primary records: _work/<run-id>/{config.json,dataset.jsonl,run.log.jsonl,summary.json}; three root comparison exports are duplicate views.

[2] PASADHI retrospective evidence audit, bench/results/fireworks-report-v1.0/analysis.json and run-summary.csv, generated for this white paper. Includes input SHA-256 fingerprints, canonical-request counts and per-run checks.

[3] PASADHI load-test harness, app/hardtest.py, current source inspected 28 October 2026. Relevant routines: execute_one, _execute_sim, _resource_sample, run and build_summary. Historical run-time source revision is unavailable.

[4] PASADHI Fireworks adapter, app/fireworks.py, FireworksBackend.complete, current source inspected 28 October 2026. Contains the local 16,000-character validation before HTTP dispatch and retained response-usage extraction.

[5] PASADHI deployment comparison driver, bench/compare_fireworks.py, current source inspected 28 October 2026. Defines the four prompt categories and timestamped export format.

[6] ResearchPaperTemplate, retained personal document template and visual layout specification. Used for two-column typesetting; not an official publisher acceptance or IEEE compliance certification.